

IA Aplicada com LLMs

Aula 02 — Controle da saída

Um LLM não sabe quando parar

Não é provocação, é literal.

O modelo produz uma distribuição, sorteia um token, repete. Em nenhum lugar desse laço existe:

- um contador de palavras
- um plano da resposta
- uma noção de "já falei o suficiente"

Existe um **token de fim de sequência** que, **quando sorteado**, encerra.

Se ele não for sorteado, o modelo continua — até alguém mandar parar.

Esse alguém é você

	Como	Garantia	Efeito
Instrução no prompt	"responda em até 3 frases"	nenhuma — é sugestão	molda a forma
Parâmetro da API	<code>max_tokens=150</code>	absoluta — é um corte	impede o desastre

A instrução molda. O parâmetro protege.

Só instrução → sistema que *quase sempre* funciona.

Só parâmetro → respostas cortadas no meio da frase.

Roteiro

1. Por que o modelo não controla o próprio tamanho
2. Comprimento máximo — `max_tokens`
3. Sequências de parada — `stop`
4. Penalidade de frequência e de presença
5. Quando NÃO usar penalidade
6. Os quatro juntos — receitas

Objetivos

- **Distinguir** controle por **instrução** de controle por **parâmetro**
- **Dimensionar** `max_tokens` e **detectar** truncamento
- **Projetar** sequências de parada para formatos estruturados
- **Distinguir** frequência de presença **pela fórmula**
- **Decidir** quando penalidade **não** é a resposta

Parte 1

Por que o modelo não controla o tamanho

Peça "exatamente 50 palavras"

Conte o resultado. Vai dar 43, ou 61. Quase nunca 50.

A razão é a mesma de errar ao contar letras: **o modelo opera em tokens, e não tem contador.**

Ele aprendeu que um texto "de 50 palavras" tem uma certa **aparência**, e produz algo com essa aparência.

É imitação de forma, não aritmética.

O que funciona e o que não funciona

✓ **A forma** — "seja breve", "um parágrafo", "em tópicos"

✗ **O número** — "exatamente 50 palavras", "no máximo 300 caracteres"

Instrução de tamanho é **estatística**, não contrato.

Três formas de encerrar uma geração

geração token a token

- token de fim sorteado —> finish_reason = "stop"
(o MODELO decidiu)
- sequência de stop —————> finish_reason = "stop"
(VOCÊ marcou)
- max_tokens atingido —————> finish_reason = "length"
(VOCÊ cortou)

Duas das três são suas. O modelo só oferece a primeira.

Parte 2

Comprimento máximo

Para que serve

`max_tokens` = teto de tokens **gerados** (a entrada não conta).

1. **Teto de custo por chamada** — token de saída é a tarifa cara. É o único jeito de garantir que uma chamada não custe mais que X
2. **Teto de latência** — resposta com teto tem tempo com teto
3. **Proteção contra geração desgovernada** — modelos entram em laços de repetição, especialmente com temperatura alta

O que ele NÃO é

`max_tokens` não pede uma resposta mais curta.

É uma **guilhotina**.

O modelo **não sabe** qual é o seu `max_tokens`. Não planeja a resposta para caber no orçamento.

Escreve como escreveria de qualquer jeito — e a geração é **interrompida** no meio da frase, da palavra, do JSON.

Errado × certo

```
# ERRADO — espera que o parâmetro encurte a resposta
messages=[{"role": "user", "content": "Explique o que é RAG."}],
max_tokens=50,          # resultado: metade de uma explicação longa

# CERTO — a instrução molda; o parâmetro protege
messages=[{"role": "user",
            "content": "Explique o que é RAG em no máximo 2 frases."}],
max_tokens=120,        # folga sobre o esperado, como rede de segurança
```

Se o `max_tokens` está sendo atingido com frequência, ele não está **protegendo** — está **censurando**.

Como dimensionar

1. **Meça o caso médio** — 10 execuções, olhe `usage.completion_tokens`
2. **Estime o pior caso** — $\sim 2\times$ a mediana; em JSON, calcule o maior que o schema permite
3. **Some folga** — e lembre: **português usa $\sim 46\%$ mais tokens** que inglês.
Teto calibrado com exemplos em inglês **trunca em produção**
4. **Monitore a taxa de** `finish_reason="length"` — subindo, é aviso antecedente de que o dado cresceu

Em saída estruturada, é pior

Um JSON truncado **não** é um JSON parcialmente útil.

É um **erro de sintaxe**.

Se o corte veio no meio, o `json.loads` estoura e você não recupera nada:

| pagou todos os tokens e não levou nada.

Com `response_format`, `max_tokens` apertado é mais perigoso que em texto livre.

Parte 3

Sequências de parada

O que são

`stop` recebe uma lista de textos. Ao gerar qualquer um, a geração **para imediatamente** — e a sequência não aparece na saída.

```
stop=["Atenciosamente", "Cordialmente"]
```

	Corta por
<code>max_tokens</code>	orçamento (quantidade)
<code>stop</code>	conteúdo (semântica)

"pare depois de 200 tokens" × "pare quando chegar na despedida"

O caso do e-mail

Você pede um e-mail comercial. O modelo entrega o e-mail — **mais** a assinatura, **mais** um "espero que isso ajude!", **mais** uma oferta de ajuda adicional.

Nada disso vai para o cliente.

Com `stop=["Atenciosamente", "Cordialmente"]` :

- o texto sai **no formato que você precisa** — a assinatura vem do **código**, com o nome certo do remetente
- **você não paga** pelos tokens do fechamento
- a resposta chega **mais rápido**

Formatos que pedem **stop**

Formato	stop típico	Por quê
E-mail	<code>["Atenciosamente"]</code>	corta a despedida
Lista de 3 itens	<code>["\n4."]</code>	evita continuar além do pedido
Diálogo	<code>["Usuário:", "Cliente:"]</code>	impede inventar a fala do outro lado
Bloco de código	<code>["``"]</code>	corta o comentário depois do código
Um parágrafo	<code>["\n\n"]</code>	a quebra dupla marca o fim

O caso do diálogo merece destaque

Sem `stop`, o modelo às vezes escreve:

- a resposta do assistente
- **e** a próxima pergunta do usuário
- **e** a resposta seguinte

...simulando uma conversa inteira sozinho.

Não é bug. Ele foi treinado para continuar texto, e o texto que ele viu tem os dois lados.

`stop=["Usuário:"]` resolve.

Três cuidados

1. **A sequência pode aparecer no meio do conteúdo.** `stop=["3."]` numa lista e o item 1 menciona "R\$ 3.500" → a geração morre ali. Escolha sequências que **não podem** ocorrer no conteúdo.
2. `finish_reason` **fica ambíguo.** Em muitos provedores, parar por `stop` também devolve `"stop"` — indistinguível do término natural.
3. `stop` **não conserta prompt ruim.** Se o modelo insiste na despedida, o prompt não deixou claro o formato. `stop` é a rede; o prompt é o combinado.

Parte 4

As duas penalidades

Onde elas agem

Ambas agem **depois** dos logits e **antes** do sorteio: subtraem valor dos logits de tokens **já usados**.

Frequência — desconto proporcional à **contagem**:

$$z'_i = z_i - \alpha \cdot c_i$$

Presença — desconto **fixo**, se já apareceu:

$$z'_i = z_i - \beta \cdot 1[c_i > 0]$$

A diferença, em números

Com $\alpha = \beta = 0,5$, desconto aplicado ao logit:

Vezes que já apareceu	Frequência	Presença
0	0	0
1	-0,5	-0,5
2	-1,0	-0,5
3	-1,5	-0,5
5	-2,5	-0,5

Lendo a tabela

Presença é um empurrão **único**, na primeira reincidência:

"você já falou disso; explore outra coisa."

Depois, não aumenta a pressão.

Frequência é pressão **crescente**:

"cada vez que você repete, fica mais caro repetir de novo."

A divisão de trabalho

	Combate	Use quando
<code>frequency_penalty</code>	repetição literal e insistente	o texto martela os mesmos termos
<code>presence_penalty</code>	permanecer no mesmo assunto	as ideias giram em círculos — típico de brainstorm

Frequência controla o **texto**.

Presença controla o **assunto**.

A faixa de valores

Tipicamente **-2 a 2**:

Valor	Efeito
0	desligada (padrão)
0,1 – 0,5	correção suave, quase invisível
0,5 – 1,0	o vocabulário se abre
> 1,0	agressiva — começa a evitar palavras necessárias
negativo	encoraja repetição

O valor negativo é o melhor experimento da aula: `frequency_penalty=-1.5` e veja o texto travar em laço. Prova que o parâmetro age sobre os logits.

Parte 5

Quando NÃO usar penalidade

Penalidade é um instrumento cego

Ela penaliza **tokens**, sem saber se aquele token era decorativo ou essencial.

E existem tokens que a sua saída **precisa** repetir.

Nunca use penalidade em...

Caso	Por quê
JSON, XML, código	as chaves repetem por construção; { , " , : aparecem dezenas de vezes → quebra o formato
Terminologia técnica	"hemoglobina glicada" tem que repetir. Com penalidade, o modelo inventa sinônimos — e sinônimo inventado em laudo é erro factual
Extração / classificação	saída curta e determinada: nada a combater, só risco
Tabelas e listas	os rótulos repetem de propósito

Antes do botão, o prompt

```
# Caminho 1 – girar o botão
gerar(prompt="Escreva sobre nossa cafeteria.",
      frequency_penalty=1.2)

# Caminho 2 – dizer o que você quer
gerar(prompt="\"Escreva 3 parágrafos sobre nossa cafeteria.
Cada parágrafo trata de um aspecto: o café, o ambiente, o atendimento.
Não repita a palavra \"café\" mais de uma vez por parágrafo.\"")
```

O caminho 2 é mais confiável, mais legível, mais fácil de depurar, e **sem efeito colateral**.

A penalidade é o último recurso, não o primeiro.

Parte 6

Os quatro juntos

Receitas

Caso	max_tokens	stop	freq	pres
E-mail	400	["Atenciosamente"]	0	0
Chat	300	—	0	0
Extração / JSON	pior caso do schema	—	0	0
Lista de 3	200	["\n4."]	0	0 – 0,3
Diálogo	200	["Usuário:"]	0 – 0,3	0
Brainstorm	500	—	0 – 0,3	0,3 – 0,6
Código	800	["` ``"]	0	0

Repare na coluna de penalidades

Ela é **quase toda zero**.

Não é descuido. É o resumo honesto da Parte 5.

Exemplo: o e-mail que para na despedida

```
for rotulo, extras in [  
    ("sem stop", {}),  
    ("com stop", {"stop": ["Atenciosamente", "Cordialmente"]}),  
]:  
    r = client.chat.completions.create(  
        model=MODELO, messages=[{"role": "user", "content": PEDIDO}],  
        temperature=0.4, max_tokens=400, **extras)  
    print(rotulo, r.usage.completion_tokens, r.choices[0].finish_reason)  
  
# A assinatura é responsabilidade do CÓDIGO:  
ASSINATURA = "\n\nAtenciosamente,\nCentral de Atendimento – XYZ\n(11) 4000-0000"
```

Menos tokens e mais confiável ao mesmo tempo.

Síntese (1/2)

- Um LLM não sabe quando parar. Duas das três formas de encerrar são **suas**
- Instrução molda, parâmetro protege — use os dois
- O modelo **não conta palavras**: instrução de tamanho acerta a *forma*, não o *número*
- `max_tokens` é uma **guilhotina**, não um pedido de brevidade. Se é atingido rotineiramente, está censurando
- Dimensione **medindo** — e português gasta **~46% mais tokens** que inglês
- **JSON truncado é perda total**: paga tudo, não leva nada

Síntese (2/2)

- **stop** corta por conteúdo, **max_tokens** por orçamento. A sequência **não** aparece na saída
- Formatos que pedem **stop**: e-mail, lista, **diálogo**, bloco de código
- **frequency_penalty** = pressão **crescente** ($-\alpha c_i$), combate repetição literal
- **presence_penalty** = empurrão **único** ($-\beta$), combate permanência no assunto
- *Frequência controla o texto; presença controla o assunto*
- **Não use penalidade** em JSON, código, terminologia técnica, extração
- Se o texto repete, **o prompt quase sempre é a correção certa**